Embedded Data Structure
(Lecture Note 12)

Tree (1)

2025. 4. 10

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Where am I ?

Are you ready to go higher ?

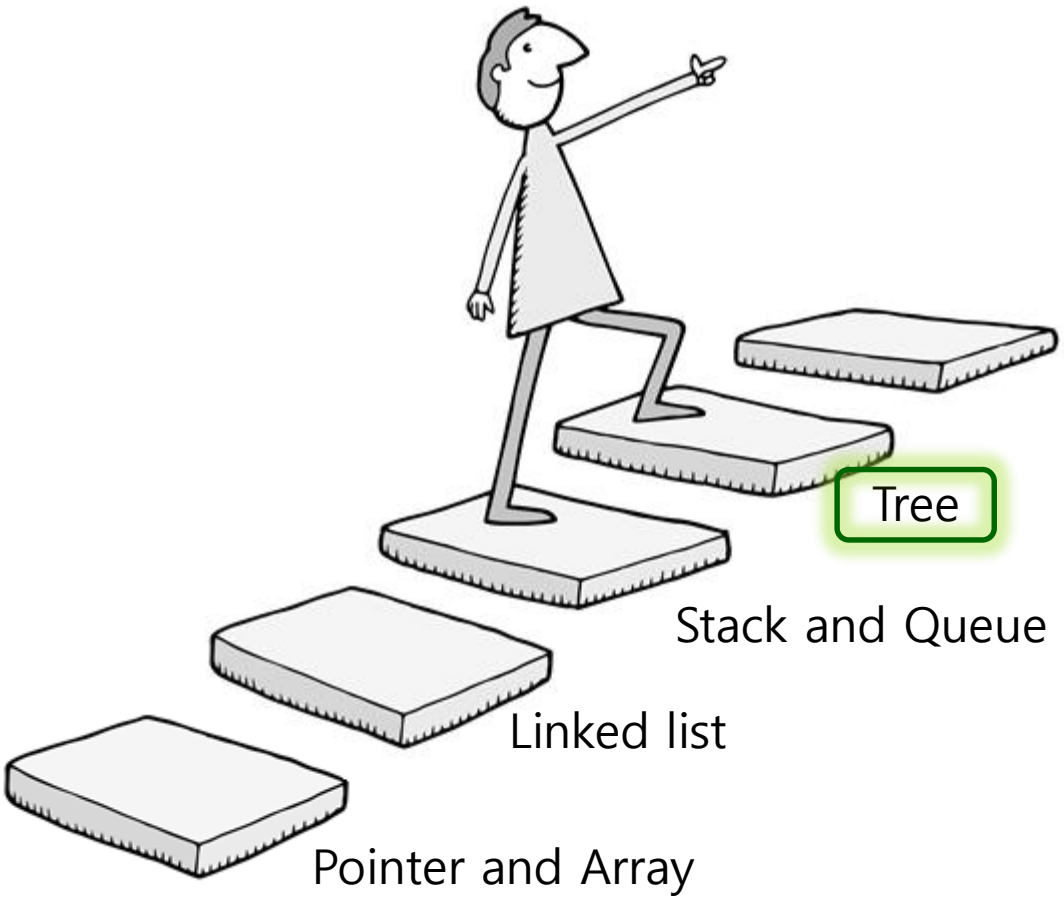


Image from http://wiki.dpconline.org/index.php?title=Step_by_step_guide_to_building_a_business_case

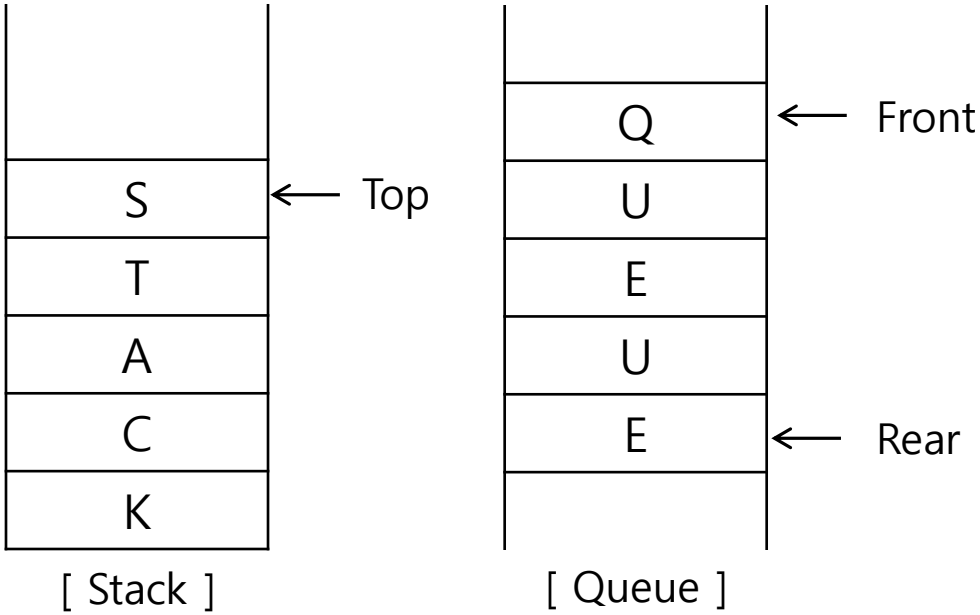
Review of Last Class (1/5)

- **Stack and queue**
  - **Various applications even with the simple difference**

! Remember the structure of stack and queue

! Practice yourself to implement stack and queue with "**linked lists**"

* What are problems by employing "arrays" for this task ?



→ What is the main function ?

→ Draw your linked list for those functions

Review of Last Class (2/5)

- **Practice : make advanced postfix conversion**
  - **In this practice, we do not consider unnecessary outer brackets**

(((1*(2+(3/4))+5)/6)+7) ➡ **(1*(2+3/4)+5)/6+7**

(Last class) (Today class)

Pseudo code : **precedence (high → low) : * or / → + or - → (**

1. Push when you meet "("
2. When you meet ")", pop and print out until "(", and discard "("
3. When you meet operators, pop and print until you meet the lower priority one. After that, push the current operator
4. When you meet the operand, just print it out
5. After the end of the equation, just pop all the things and print out

Review of Last Class (3/5)

- **Practice : make advanced postfix conversion – cont'd**
  - **Try to write down conversion results on your paper !**

Given equation : (1*(2+3/4)+5)/6+7

Char	Display	Stack (top → bottom)	Description
(		(	Push "("

Review of Last Class (4/5)

- **Practice : make advanced postfix conversion – cont'd**
  - **Three additional functions for our implementation**

```
int get_stack_top()
{
   return (top < 0) ? -1 : stack[top];
}
```

```
int is_op(int k)
{
   return (k == '+' || k == '-' || k == '*' || k == '/')';
}
```

```
int precedence(int op)
{
   if(op == '(') return 0; if(op == '+' || op == '-') return 1;
   if(op == '*' || op == '/') return 2; else return 3;
}
```

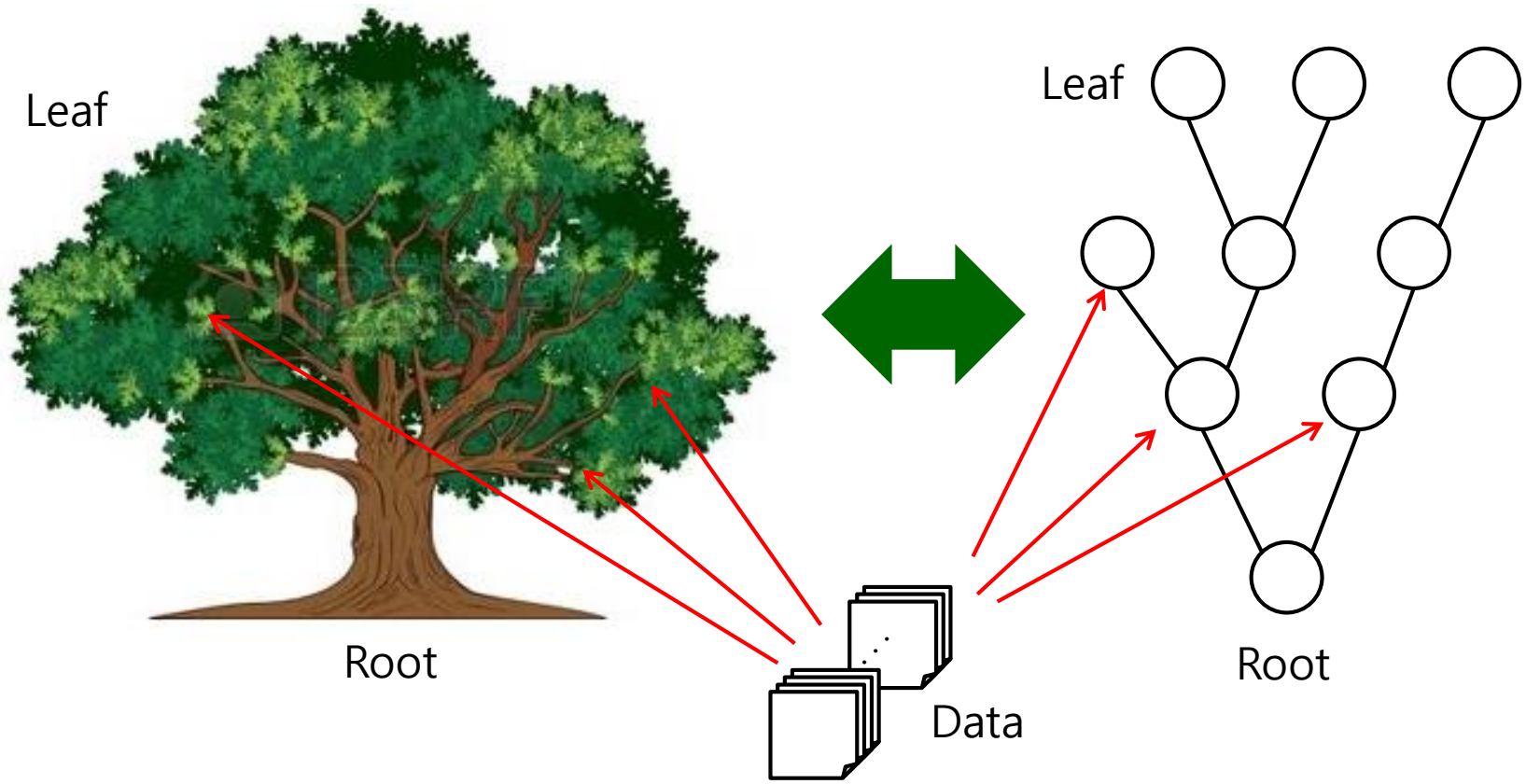
Review of Last Class (5/5)

- **Practice : make advanced postfix conversion – cont'd**

```
void postfix(char *dst, char *src)
{
    char c;
    init_stack();
    while(*src){
        if(*src == '(')
        {
            push(*src);
            src++;
        }
        else if(*src == ')') {
            while(get_stack_top() !='('){
                *dst++ = pop();
                *dst++ = ' ';
            }
            src++;
        }
        else if(is_op(*src)) {
            while(!is_stack_empty() &&
                >= precedence(*src)){
                *dst++ = pop();
                *dst++ = ' ';
            }
            src++;
        }
        else if(*src >= '0' && *src <= '9') {
            do{
                *dst++ = *src++;
            }while(*src >= '0' && *src <= '9');
            *dst++ = ' ';
        }
        else
            src++;
    }
    while(!is_stack_empty()) {
        *dst++ = pop();
        *dst++ = ' ';
    }
    dst--;
    *dst = 0;
}
```

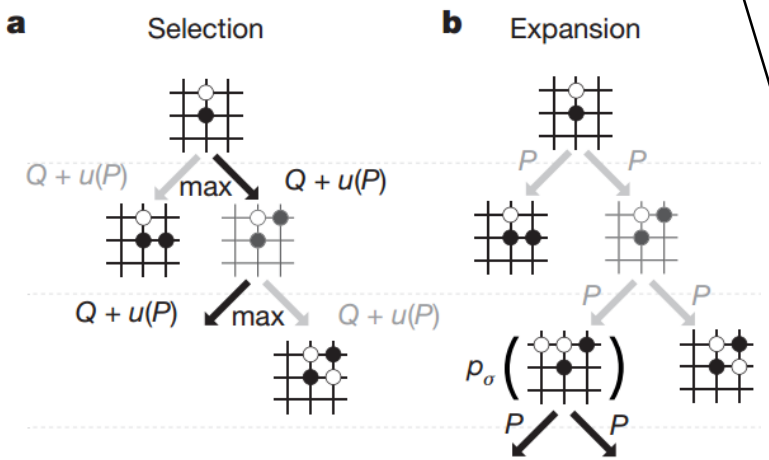
Overview (1/2)

- **Keep your focus on the structure of "tree"**
  - **Journey from root to leaf ! (node and edge)**



Overview (2/2)

- **Examples of tree-based applications**



Monte-Carlo tree search [1]

➡ **Alpha GO**

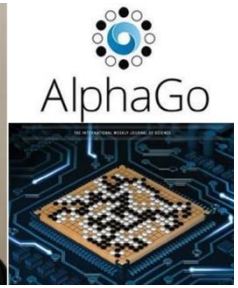
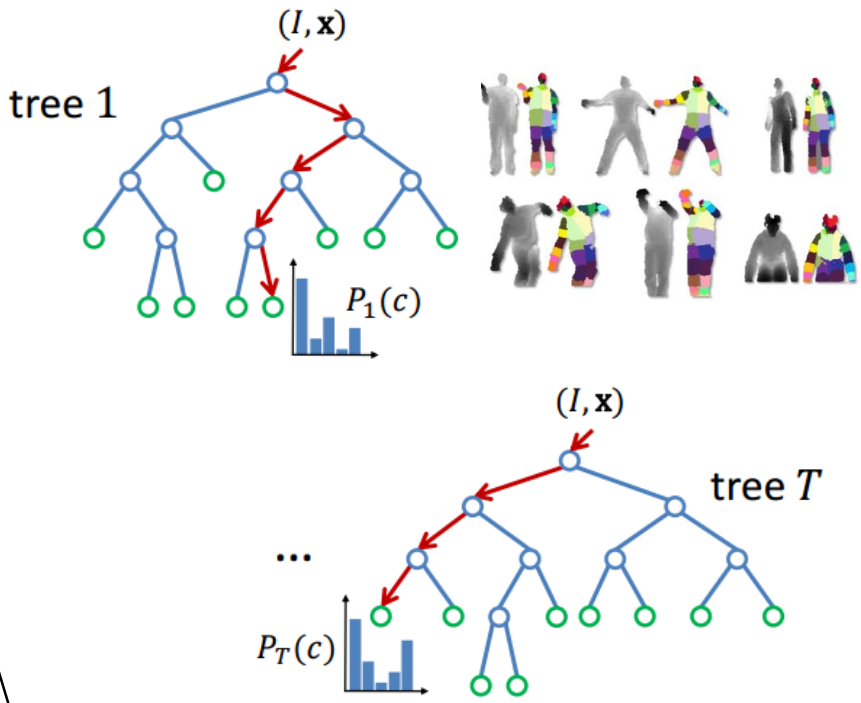


Image from Google Image



Random decision trees [2]

➡ **Microsoft Kinect**

[1] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," Nature, vol. 529, pp. 484-489, Jan. 2016.

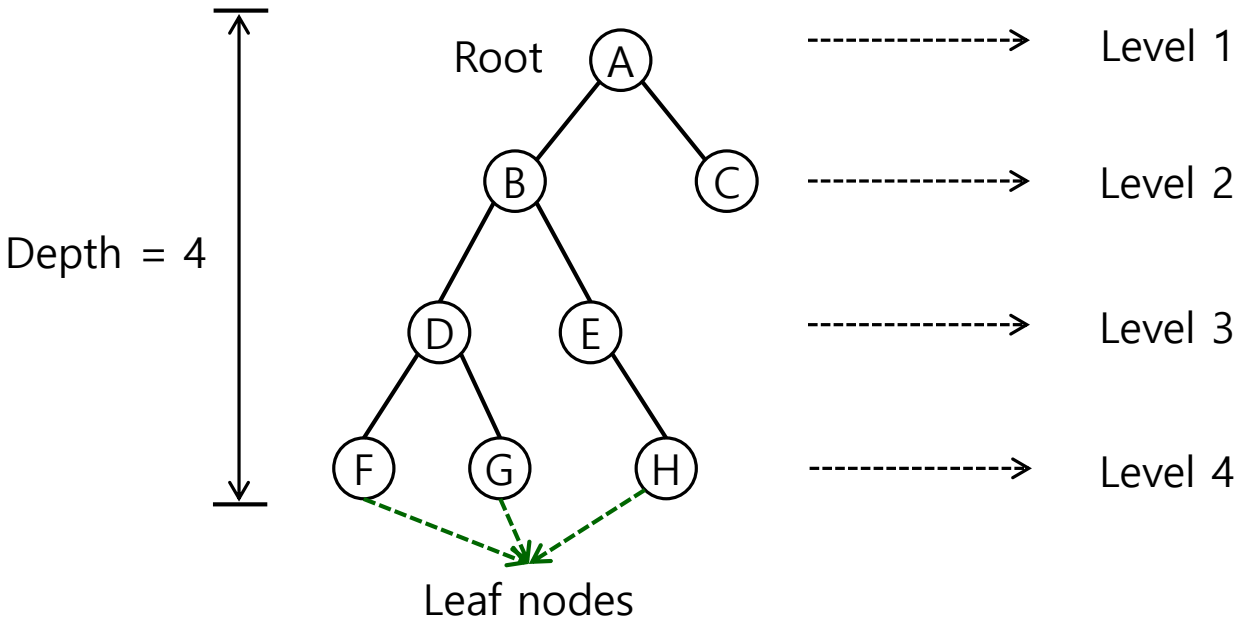
[2] J. Shotton et al., "Real-time human pose recognition in parts from single depth images," in Proc. IEEE CVPR, June 2011.

Basic concept of tree

- **2D structure means that data structure is not linear !**

➔ What does it mean ? Do we need to visit all the nodes ?

Ex) MS-DOS directory structure, organization chart, etc.

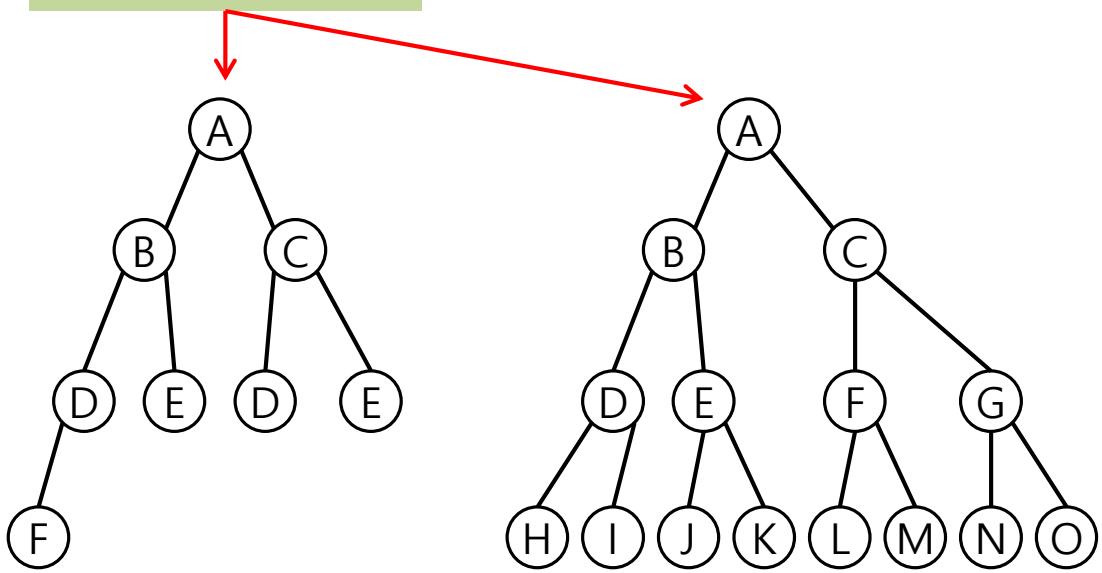


*** Note : we only consider about "binary" tree**

Introduction (2/2)

- **Characteristics of tree**
  - **One path between nodes (key difference with graph model)**
  - **N nodes yielding (N-1) links**
  - **When # of levels is d, # of nodes N can be estimated as :**

$$2^{d-1} \le N \le 2^d - 1$$



N = ? **N = ?**

Fully-connected binary tree $\lfloor \log_2 N \rfloor + 1$

$$d - 1 \le \log_2 N \le \log_2 (2^d - 1)$$

where

$$\log_2 (2^d - 1) < \log_2 (2^d) = d$$

Therefore,

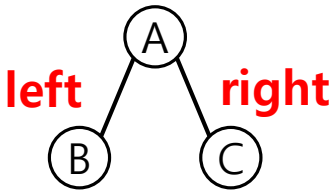
$$d \le \log_2 N + 1 < d + 1$$

Tree Representation

- **Two approaches : array-based & linked list-based**
  - **Consider about linked list-based representations**

```
typedef struct _node
{
    int key;
    struct _node *left;
    struct _node *right;
}node;

(globally defined)
node *head, *tail;
```

Start and end of tree



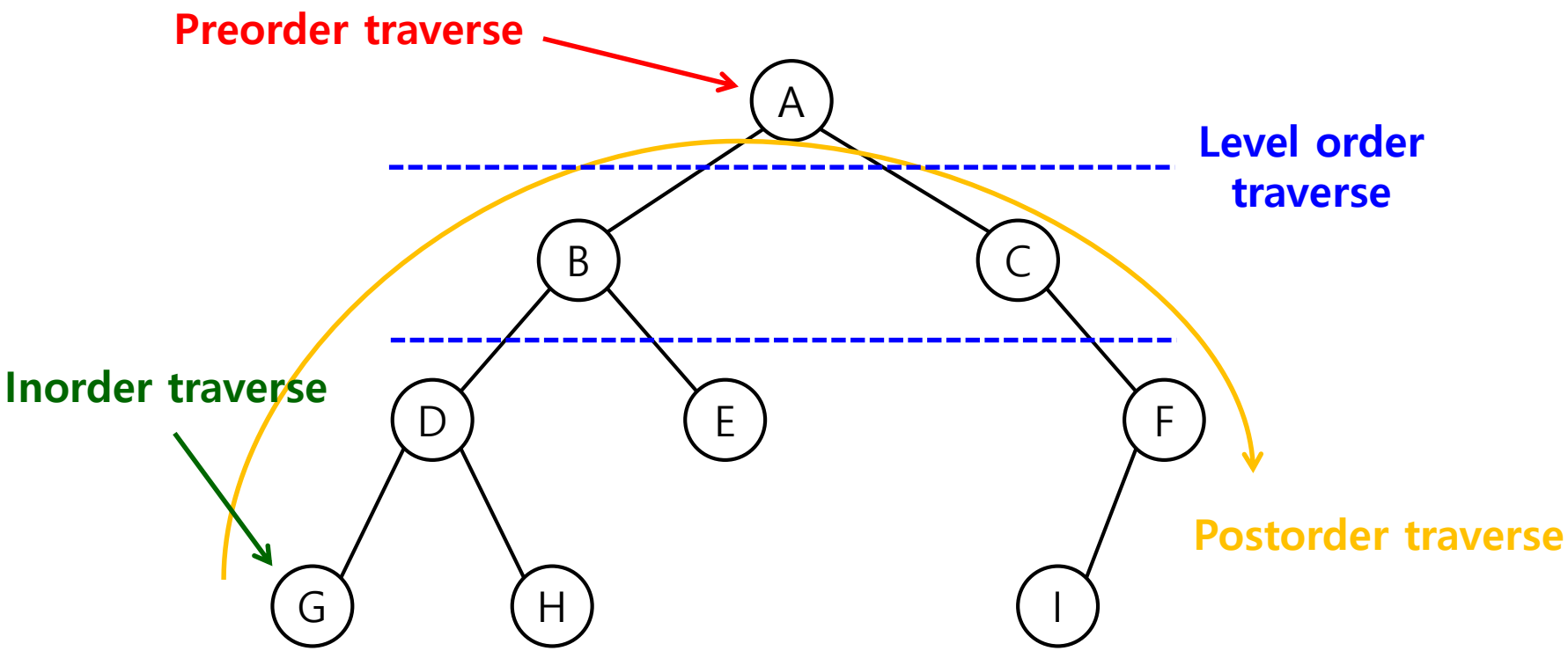

```
void init_tree()
{
    head = (node *)calloc(1, sizeof(node));
    tail = (node *)calloc(1, sizeof(node));

    head->left = tail;
    head->right = tail;
    tail->left = tail;
    tail->right = tail;
}
```

➔ **Indicate all the pointers to "tail"**

(i.e., end of the tree)

Tree Traverse

- **Brief overview of tree traverse**
  - **Visit all the nodes without overlapping**



* Tree traverse is actually implemented "recursively"

Pre-order Traverse (1/4)

- **Basic concept of pre-order traverse**

  - **Pseudo code :**

    1. Visit the root
    2. Visit the left **subtree**
    3. Visit the right subtree

Main functions :

*** Push right child node and then left one**

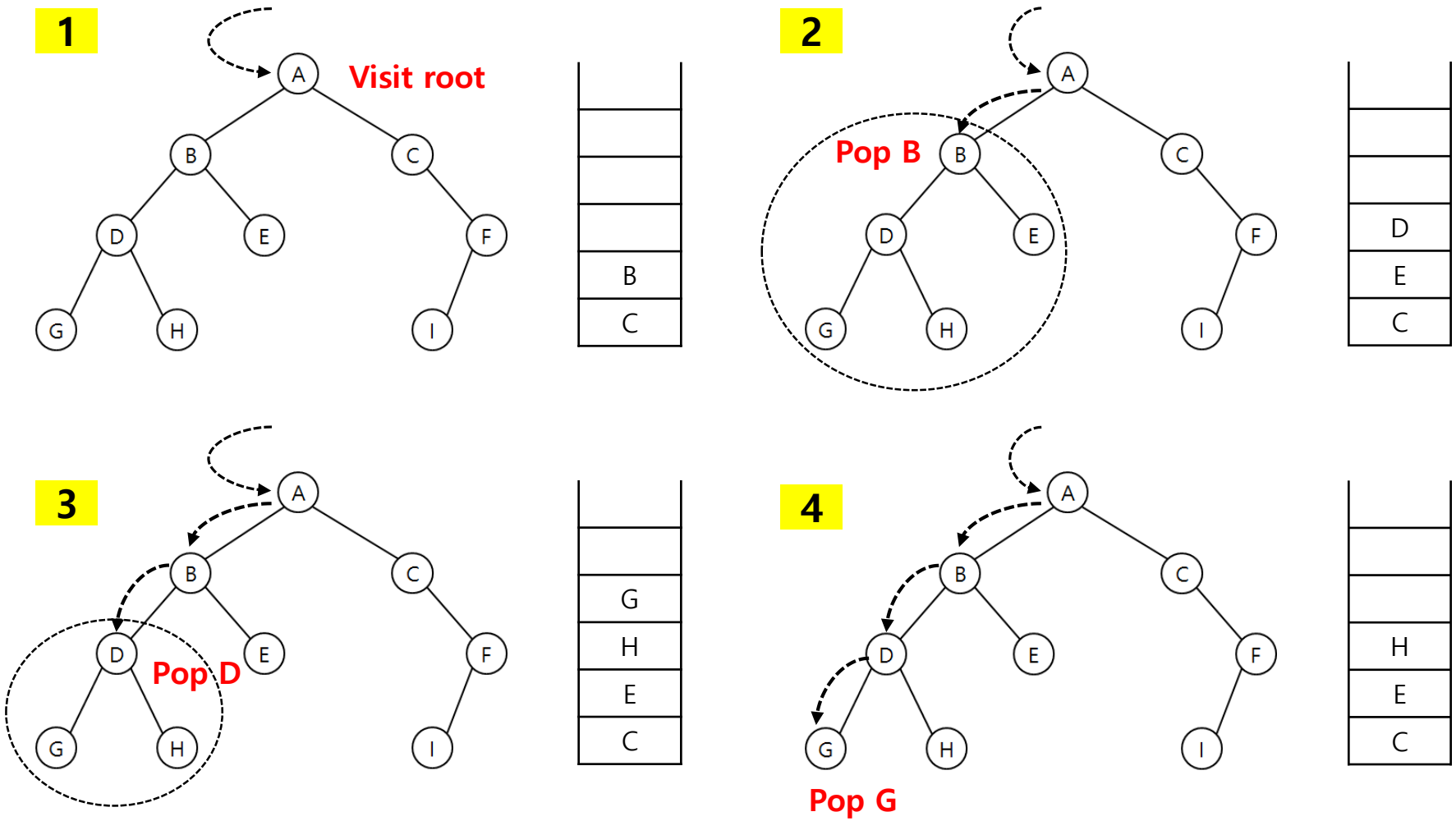
*** Pop and visit the corresponding node**

```
void preorder_traverse(node *t)
{
    if(t != tail)
    {
        visit(t);
        preorder_traverse(t->left);
        preorder_traverse(t->right);
    }
}
```

➔ **Think about "recursive" operation**

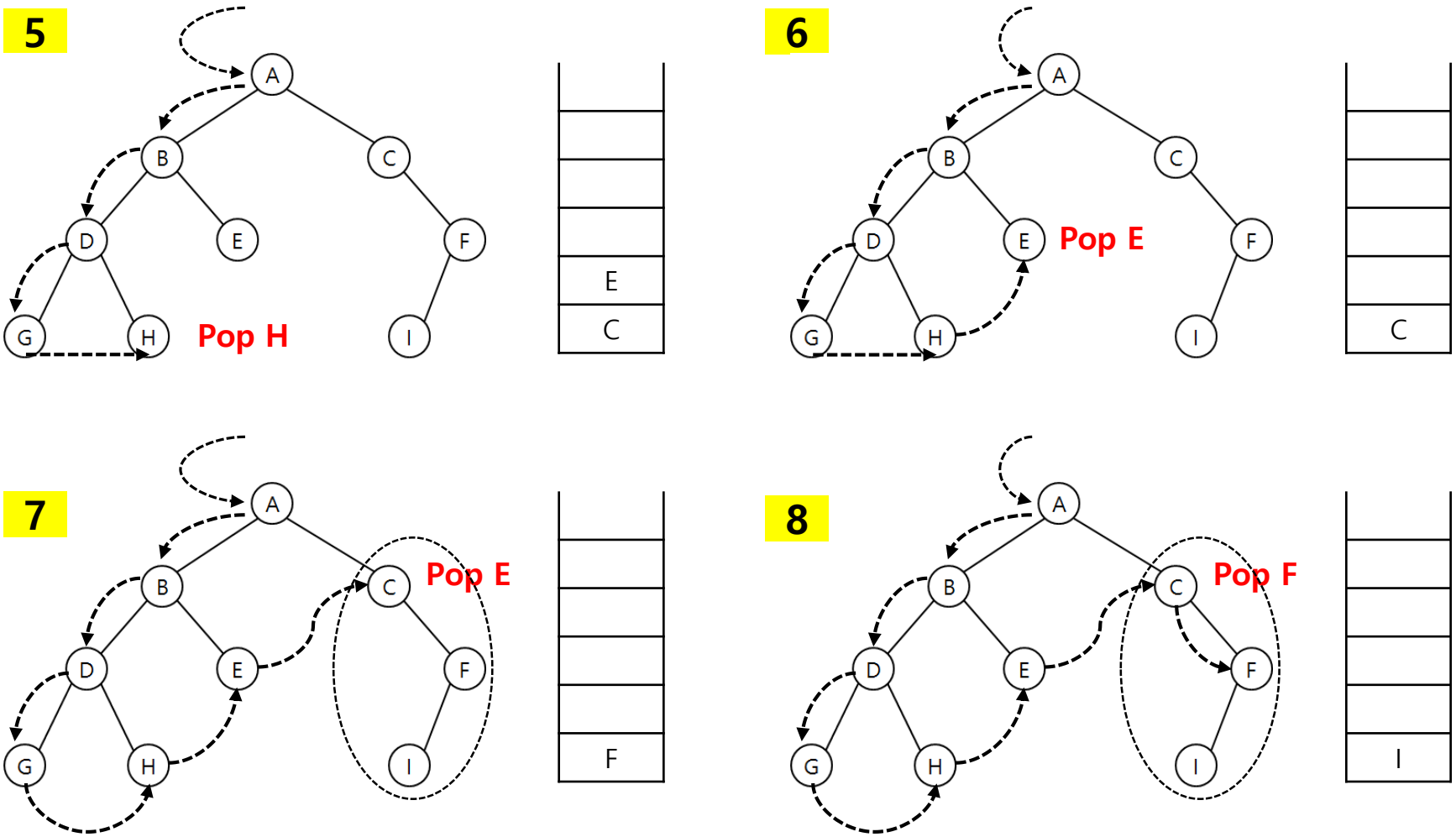
Pre-order Traverse (2/4)

- **Traverse example**



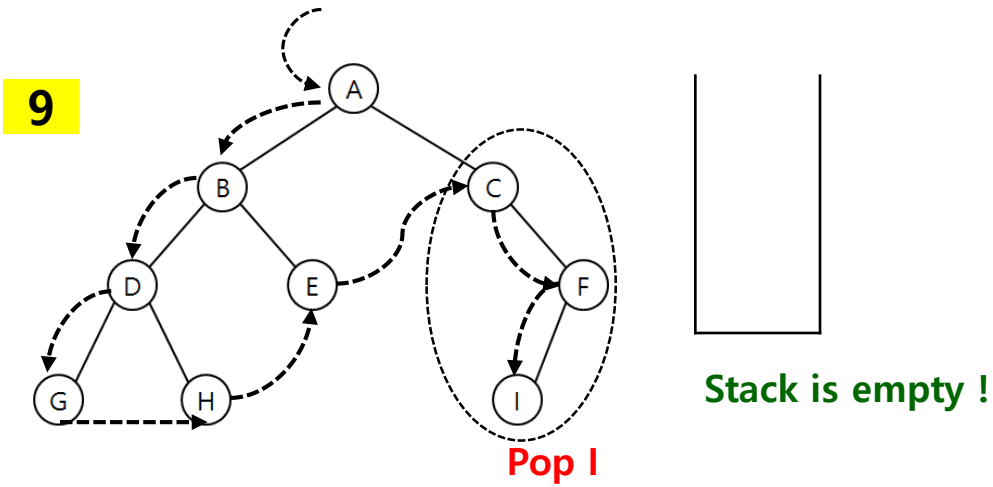
Pre-order Traverse (3/4)

- **Traverse example – cont'd**



Pre-order Traverse (4/4)

- **Traverse example – cont'd**
  - **Now, we have visited all the node**



*** Traverse result : A→B→D→G→H→E→C→F→I**

In-order Traverse (1/4)

▪ Basic concept of in-order traverse

- **Pseudo code :**

1. Visit the left subtree
2. Visit the root
3. Visit the right subtree

Main functions :

*** Push "all the left child" node**

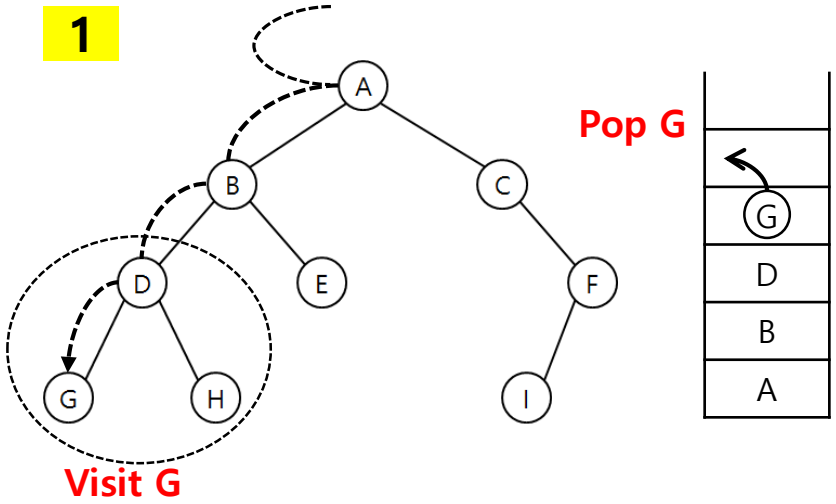
*** Pop if there is no child**

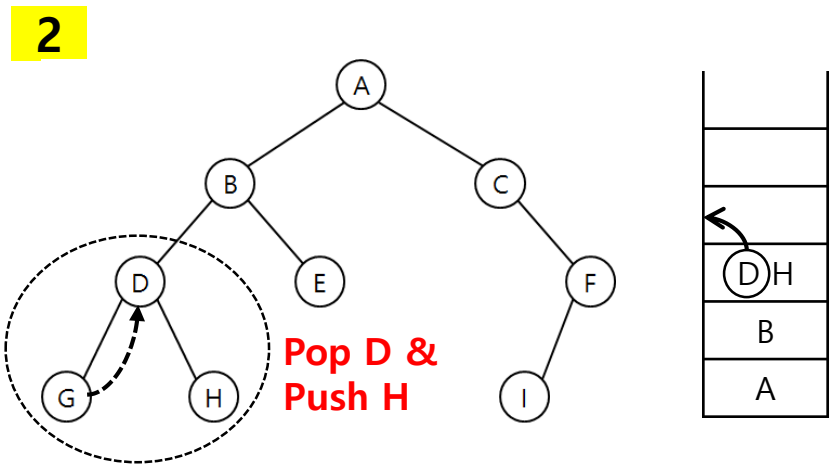
```
void inorder_traverse(node *t)
{
    if(t != tail)
    {
        inorder_traverse(t->left);
        visit(t);
        inorder_traverse(t->right);
    }
}
```

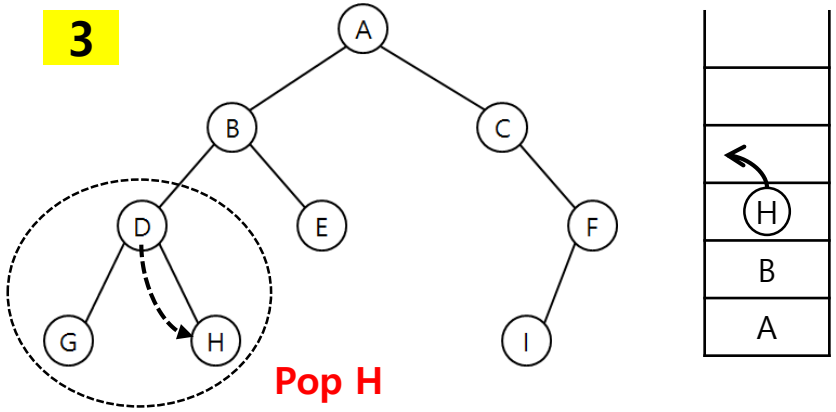
➔ Think about "recursive" operation

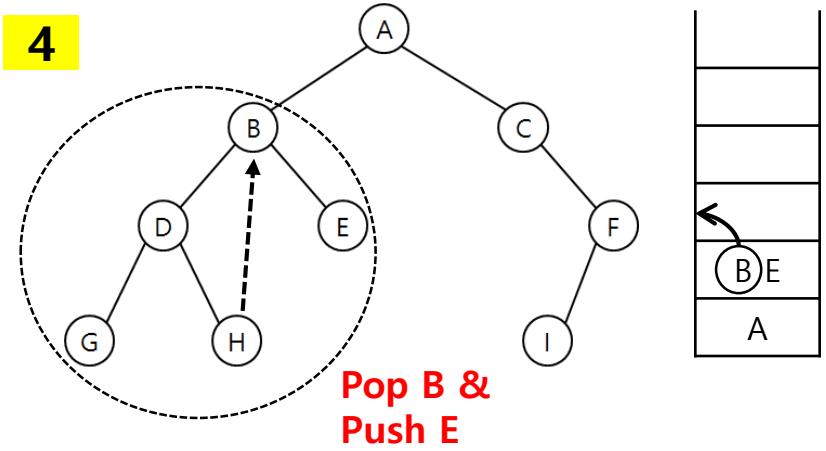
In-order Traverse (2/4)

- **Traverse example**



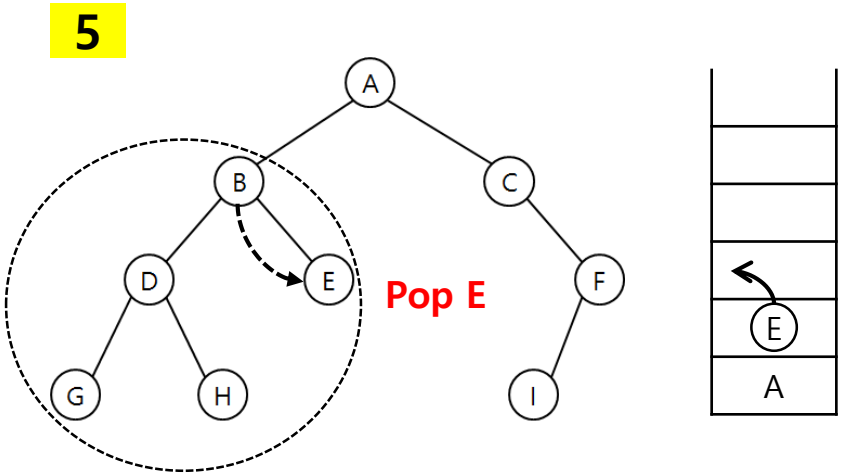


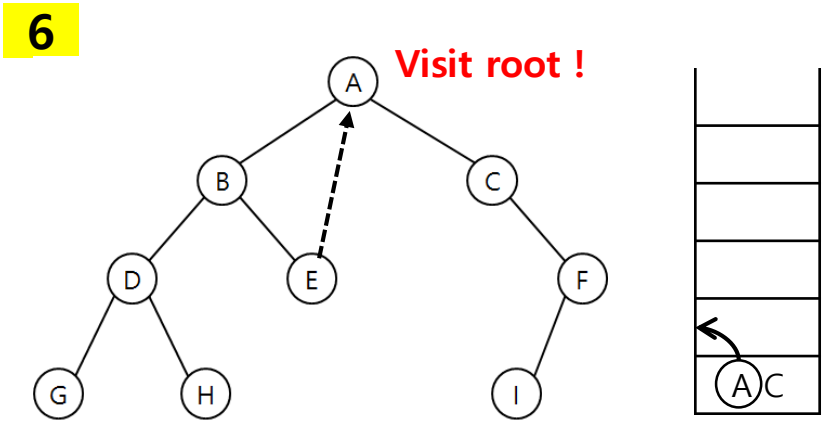


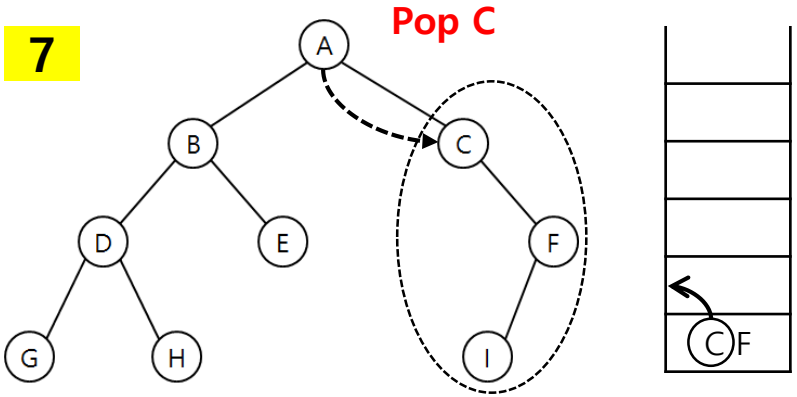


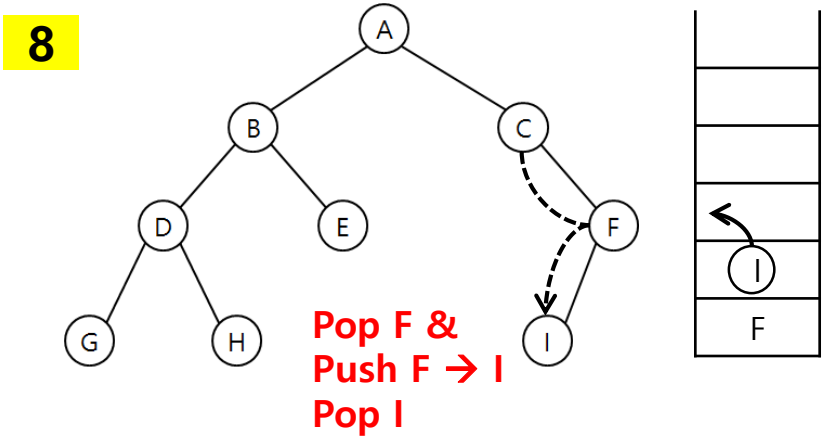
In-order Traverse (3/4)

- **Traverse example – cont'd**



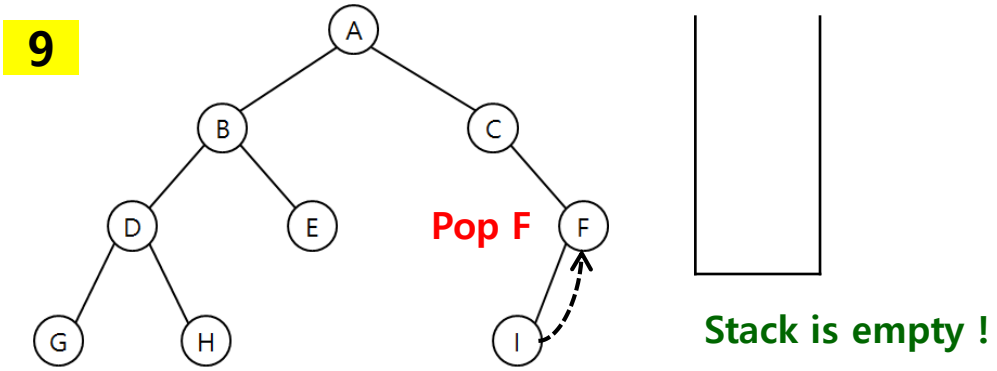






In-order Traverse (4/4)

- **Traverse example – cont'd**
  - **Now, we have visited all the node**



*** Traverse result : G→D→H→B→E→A→C→I→F**

Summary

- **Basic concept of tree and a methodology to visit it**
  - **We traverse tree with pointers (double linked lists) !**
- **Think about the applications for tree**
  - **Besides equation expression, …**